

Chapter 1 exercises — Analysis of Algorithms

23 exercises, extracted from the textbook PDF.

Caveats. (1) Statements are auto-extracted; `fi` / `fl` ligatures dropped by the PDF text layer were repaired by a cleanup pass (a rare odd gap may remain). (2) A $\triangle$ marks exercises whose statement includes a **displayed formula** that did not survive extraction — read those in the book at the cited page. The book PDF is always authoritative.

#	p.	Exercise
1.1	7	Show that $f(N) = N \lg N + O(N)$ implies that $f(N) = \Theta(N \log N)$.
1.2	9	In some situations, defining a sentinel value may be inconvenient or impractical. Implement a mergesort that avoids doing so (see Sedgwick [26] for various strategies).
1.3	9	Implement a mergesort that divides the array into three equal parts, sorts them, and does a three-way merge. Empirically compare its running time with standard mergesort.
1.4	10	Develop a recurrence describing the quantity $C_{N+1} - C_N$ and use this to prove that $\sum_{1 \leq k < N} C_k = (\lfloor \lg N \rfloor + 2) \cdot N$.
1.5	10	Prove that $C_N = N \lfloor \lg N \rfloor + N - 2 \lfloor \lg N \rfloor$.
1.6	10	Analyze the number of compares used by the three-way mergesort proposed in Exercise 1.2.
1.7	10	Assume that the running time of mergesort is $cN \lg N + dN$, where c and d are machine-dependent constants. Show that if we implement the program on a particular machine and observe a running time t_N for some value of N , then we can accurately estimate the running time for $2N$ by $2t_N(1 + 1/\lg N)$, independent of the machine.
1.8	10	Implement mergesort on one or more computers, observe the running time for $N = 1,000,000$, and predict the running time for $N = 10,000,000$ as in the previous exercise. Then observe the running time for $N = 10,000,000$ and calculate the percentage accuracy of the prediction.
1.9	12	Suppose that it is known that each of the items in an N -item array has one of two distinct values. Give a sorting method that takes time proportional to N .
1.10	12	Answer the previous exercise for three distinct values.
1.11	15	Run experiments on two different computers to test the hypothesis that the running time of mergesort divided by the number of compares that it uses approaches a constant as the problem size increases.
1.12	23	Give the recurrence for the total number of compares used by quicksort on all $N!$ permutations of N elements.
1.13	23	Prove that the subarrays left after partitioning a random permutation are themselves both random permutations. Then prove that this is not the case if, for example, the right pointer is initialized at $j := r + 1$ for partitioning.
1.14	23	Follow through the steps above to solve the recurrence $AN = 1 +$

#	p.	Exercise
1.15	24	Show that the average number of exchanges used during the first partitioning stage (before the pointers cross) is $(N - 2)/6$. (us, by linearity of the recurrences, $BN = 61 CN - 12 AN$.)
1.16	25	How many subarrays of size 2 or less are encountered, on the average, when sorting a random file of size N with quicksort?
1.17	25	If we change the first line in the quicksort implementation above to if $r - l \leq M$ then insertionsort(l, r) else
1.18	25	Ignoring small terms (those significantly less than N) in the answer to the previous exercise, find a function $f(M)$ so that the number of compares is approximately $2N \ln N + f(M)N$. Plot the function $f(M)$, and find the value of M that minimizes the function.
1.19	25	As M gets larger, the number of compares increases again from the minimum just derived. How large must M get before the number of compares exceeds the original number (at $M = 0$)?
1.20	28	How many keys in a file of 104 random integers less than 106 are likely to be equal to some other key in the file? Run simulations, or do a mathematical analysis (with the help of a system for mathematical calculations), or do both.
1.21	28	Experiment with files consisting of random positive integers less than M for $M = 10,000, 1000, 100$ and other values. Compare the performance of quicksort on such files with its performance on random permutations of the same size. Characterize situations where the random permutation model is inaccurate.
1.22	29	Discuss the idea of having a table similar to Table 1.1 for mergesort.
1.23	32	What is the standard deviation of the number of compares for the mergesort implementation given earlier in this chapter?